

Hands-On Lab | Exercise 2

JavaScript: Make the Page Say the Right Thing

 USTH  Web Application Development  4 hours  VS Code + Chrome

Learning Outcomes

By the end of this session, you will be able to:

- Write a result onto the page instead of hiding it in the Console, and choose between `textContent` and `innerHTML` on purpose.
- Explain what `NaN` and `Infinity` are, and why a page shows them instead of stopping.
- Read a red error in the Console and trace it back to the line that caused it.
- Say why a form reloads the page, and stop it.
- Select elements with `querySelectorAll` and know the difference a single character makes.
- Load a file with `fetch` and explain why the same code works over Live Server but not by double-clicking.

Setup

Editor: VS Code with the **Live Server** extension.

Browser: Chrome or Firefox - you will need `F12`.

Files from your instructor: `order-total.html` (Part 1), `signup.html` (Part 2), `tasks.html` + `menu.json` (Part 3).

Rule for every part: the answer must appear *on the page*. The Console is for reading errors, not for showing results to a user.

Part 1 Make the Page Lie to You (40 min)

Open `order-total.html` with Live Server. It works. Then feed it the three sets of input below and watch what the page prints.

Each set makes the page display something that is not what the user meant. Your job is to say **what appeared**, **why**, and **how you would fix it**.

1.1 `order-total.html`

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Order total</title>
</head>
<body>

  <h2>Order total</h2>

  <div><label>Item name:</label> <input type="text" id="item" placeholder="Pho bo"></div>
  <div><label>Price:</label> <input type="number" id="price" placeholder="60000"></div>
  <div><label>Quantity:</label> <input type="number" id="qty" placeholder="3"></div>

  <div>
    <button onclick="total()">Total</button>
    <button onclick="perPerson()">Split per person</button>
  </div>

  <div id="result">The result is:</div>

  <script>
    function total() {
      const name = document.getElementById("item").value;
      const price = parseFloat(document.getElementById("price").value);
      const qty = parseFloat(document.getElementById("qty").value);

      if (isNaN(price) || isNaN(qty)) {
        document.getElementById("result").innerHTML = "Invalid input";
      }

      const sum = price * qty;
      document.getElementById("result").innerHTML = qty + " x " + name + " = " + sum;
    }

    function perPerson() {
      const name = document.getElementById("item").value;
      const price = parseFloat(document.getElementById("price").value);
      const qty = parseFloat(document.getElementById("qty").value);

      const each = price / qty;
      document.getElementById("result").innerHTML = name + ": " + each + " per person";
    }
  </script>

</body>
</html>
```

1.2 Three runs

Toolbox

Values: NaN Infinity **Functions:** parseFloat isNaN

DOM: .value innerHTML textContent

Concepts: an if without return keeps going; the browser parses whatever you hand to innerHTML

Q1. Item name `Pho bo` , **Price** left empty, **Quantity** `3` , press **Total**.

Watch the result line closely, then answer: what is printed? The code contains a line that writes `Invalid input` - did you ever see those two words on screen? Explain what happened to them. What single word, added in one place, would fix this?

Q2. Item name `Pho bo` , **Price** `60000` , **Quantity** `0` , press **Split per person**.

What is printed? In most languages dividing by zero stops the program - here the page keeps going and prints something. What is that something, and what is its type? Now write down what the user should have seen instead.

Q3. Item name `<b>Pho bo</b>` , **Price** `60000` , **Quantity** `3` , press **Total**.

Compare the text on screen with the text you typed. They are not the same - describe the difference. Then change the two `innerHTML` in `total()` to `textContent` , reload, and run the same input again. What is printed now?

Finally: if `textContent` is the safer one, why does `innerHTML` exist at all? Give one case where `innerHTML` is the right choice.

The name of what you just saw

When a page takes text from a user and hands it to `innerHTML` , the browser does not display that text - it **runs** it as HTML. You made it print bold text. Someone else can make it run a `<script>` . This is called **XSS**, and you will meet it again in a security course.

The working rule for this course: **use** `textContent` . Reach for `innerHTML` only when the string is markup that you wrote yourself.

Part 2 Fix the Sign Up Page (60 min)

`signup.html` is the form from the old exercise sheet: type a password twice, press Submit, and the page should say whether the two match. It opens without complaint and nothing about it looks broken. It is broken in **four** places.

Before you start: keep the Console log

Press `F12` and open the **Console** tab. Click the gear icon and tick **Preserve log**.

Without this, a page that reloads wipes the Console clean - the error flashes past and you never see it. One of the four faults does exactly that.

2.1 `signup.html`

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8">
5   <title>Sign Up</title>
6   <script>
7     const box = document.getElementById("message");
8
9     function checkPassword() {
10       const pw1 = document.getElementById("pw1").value;
11       const pw2 = document.getElementById("pw2").value;
12
13       if (pw1 = pw2) {
14         box.textContent = "Passwords match";
15       } else {
16         box.textContent = "Passwords do not match";
17       }
18     }
19
20     document.addEventListener("DOMContentLoaded", function () {
21       document.getElementById("signup-form")
22         .addEventListener("blur", function (e) {
23           document.getElementById("hint").textContent =
24             "You left the " + e.target.id + " field";
25         });
26     });
27   </script>
28 </head>
29 <body>
30
31   <h2>Sign Up</h2>
32
33   <form id="signup-form" onsubmit="checkPassword()">
34     <div><label>Username:</label>      <input type="text"      id="user"></div>
35     <div><label>Password:</label>      <input type="password" id="pw1"></div>
36     <div><label>Re-type Password:</label> <input type="password" id="pw2"></div>
37     <button type="submit">Submit</button>
38   </form>
39
40   <div id="message"></div>
41   <div id="hint"></div>
42
43 </body>
44 </html>
```

2.2 What to do

Toolbox

Loading: `<script>` in `<head>` vs before `</body>` `defer` `DOMContentLoaded`

Forms: `onsubmit` `event.preventDefault()`

Operators: `=` `==` `===`

Events: which events bubble and which do not - `blur` vs `focusout`

Two of the four faults announce themselves in the Console. Two are silent: the page simply never does something it was supposed to do. Find all four.

For each one, fill in the table. The **Symptom** column is the one that carries the marks - describe what a person sitting in front of the screen would notice, before you say what the cause is.

#	Line	Symptom on screen	Cause	Fix
1				
2				
3				
4				

2.3 Four questions

- Q1.** One fault makes the whole script useless before the user touches anything. Line 7 runs the moment the browser reads it. What is in `box` at that moment, and why? Give **two** different one-line fixes - one that changes where the `<script>` sits, one that changes when line 7 runs.
- Q2.** Press Submit with two identical passwords, then with two different ones. The message is the same both times. Which line causes that, and what does that line actually do to `pw1`?
- Q3.** The hint under the form is supposed to appear when you click into a field and then click away. It never appears, and the Console stays clean. The listener is attached correctly and the function is correct - so why is it never called? Name the property of the `blur` event that is responsible, and name the event you would use instead.
- Q4.** After you fix everything, the page still reloads and blanks itself when you press Submit. What does a `<form>` do by default when it is submitted, and what one line inside `checkPassword` stops it? Why did **Preserve log** matter while you were hunting this one?

Part 3 Write `app.js` Without Touching the HTML (90 min)

🚫 The one rule

`tasks.html` is finished. **You may not change a single character of it.** Everything you write goes into `app.js`, which is already linked at the bottom of the page.

Every task must put its result **on the page**, in the element the HTML has already provided for it.

3.1 `tasks.html`

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>JavaScript tasks</title>
  <style>
    .panel    { border: 1px solid #999; padding: 10px; margin: 12px 0; }
    .highlight { background-color: gold; }
  </style>
</head>
<body>

<!-- ===== TASK 1 ===== -->
<section class="panel">
  <h2>Task 1 - Highlight</h2>

  <div>
    <h1>Introduction</h1>
    <section>
      <h1>Buried inside a section</h1>
    </section>
  </div>

  <div>
    <h1>About HTML</h1>
    <h1>About CSS</h1>
  </div>

  <h1>About JavaScript</h1>

  <button id="highlight-btn">Highlight</button>
  <p id="t1-count"></p>
</section>

<!-- ===== TASK 2 ===== -->
<section class="panel">
  <h2>Task 2 - Click timer</h2>
  <p id="target">Click me twice</p>
  <p id="t2-log"></p>
</section>

<!-- ===== TASK 3 ===== -->
<section class="panel">
  <h2>Task 3 - Load the menu</h2>
  <button id="load-btn">Load menu</button>
  <ul id="menu-list"></ul>
  <p id="t3-status"></p>
</section>

<script src="app.js"></script>
</body>
</html>
```

`menu.json` sits in the same folder:

```
[
  { "dish": "Pho bo", "price": 60000 },
```

```
{ "dish": "Pho ga", "price": 55000 },
  { "dish": "Pho tai", "price": 65000 }
}
```

3.2 The tasks

T1. Highlight the right headings. When **Highlight** is pressed, every `<h1>` that is a *direct child* of a `<div>` gets the class `highlight`. Write how many were found into `#t1-count`.

Why: run your code with the selector `"div h1"` and then with `"div > h1"`. The counts differ - say which heading is responsible and why. Then answer: the old exercise sheet asked for "h1 elements that are children of a div". Which of the two selectors matches that wording, and what does the other one mean?

T2. Time two clicks. Clicking `#target` the first time records the moment. Clicking it a second time writes the gap in milliseconds into `#t2-log`. The third click starts a fresh pair, the fourth reports again, and so on.

Why: your variable holding the first moment cannot live inside the click handler. Explain what would happen if it did. Then explain what has to happen to that variable after you report a gap - and what the page would report on clicks 3, 4 and 5 if you forgot that step.

T3. Load the menu from a file. When **Load menu** is pressed, `fetch menu.json`, then add one `<li>` per dish into `#menu-list` showing the dish and its price. Put `Loading...` into `#t3-status` while it runs, and clear it when the list appears.

Why: this one has a trap that has nothing to do with your code. Do it in this order and record what happens each time:

- Close Live Server. Open `tasks.html` by double-clicking it in your file manager. Look at the address bar, then press the button. Copy the red error from the Console word for word.
- Now open the same file with Live Server. Look at the address bar again. Press the button.

Then answer: the file did not change and your code did not change, so what *did* change? The error mentions an origin of `null` - what does that mean for a page opened by double-clicking? And who refused to serve the file: the browser, or the server?

💡 Why Task 3 matters more than it looks

In the first lecture you were told to use Live Server rather than double-clicking the file, and the reason given was that `http://localhost:5500` is a real web address while `file:///...` is not. Task 3 is where that stops being a piece of advice and becomes the difference between a page that works and a page that does not.

The rule the browser is applying is called the **same-origin policy**, and the error it prints is a **CORS** error. You will meet both again as soon as your JavaScript starts talking to a server that someone else owns.

🤖 A note on AI

Nothing here forbids you from asking an AI. But an AI cannot see the word `Infinity` sitting on your screen, it cannot tell you which of your four faults you have already fixed, and it cannot explain to your teacher why you chose `textContent`. The point of the session is that when you do use an AI - next week, at work - you can tell in ten seconds whether what it gave you is right.